Google gen-AI

Rev. 1.0 del 20/01/2026

Dal 2025 Google ha creato una nuova Developer Console dedicata all'intelligenza artificiale da cui si può creare e scaricare una apposita API KEY da utilizzare nelle proprie applicazione. Al momento il servizio è gratuito, però occorre comunque identificarsi mediante una carta di credito. Al superamento della soglia di utilizzo (max 60 richieste al minuto) non ci sarà nessun addebito automatico, ma verranno bloccate le richieste per il tempo necessario.

- Accedere a ***aistudio.google.com***
- In basso a sinistra ***GET API KEY***
- Verrà generata la chiave ed insieme viene creato anche un nuovo progetto
- In alternativa si poteva creare un nuovo progetto specifico dalla finestra PROGETTI e poi associargli una API KEY
- All'interno della dashboard si può accedere alla API KEY sia da **Progetti** sia da **Chiavi API**

La chiave può semplicemente essere copia incollata all'interno del proprio progetto

Utilizzo con nodejs / express

```
npm install @google/genai
```

```
import { GoogleGenAI } from "@google/genai";
```

```
const genAI = new GoogleGenAI({apiKey: process.env.GenAI_KEY});
```

genAI sta per Generative Artificial Intelligence, cioè Intelligenza Artificiale Generativa che, a differenza dell'IA "tradizionale", è progettata per creare nuovi contenuti: Testo (come i modelli GPT di OpenAI o Gemini di Google), Immagini(DALL-E, Midjourney), Codice (GitHub Copilot), Audio e Video.

Il nome del pacchetto @google/genai è stato scelto da Google per distinguerlo dalle altre librerie di IA dedicate alla visione artificiale (come MediaPipe) o al machine learning classico (come TensorFlow).

```
app.post('/api/chat', async (req, res) => {
   const prompt = req.body.prompt;
    if (!prompt) {
        return res.status(400).send("Inserisici una domanda ");
    }
    try {

        const response = await genAI.models.generateContent({
            model: "gemini-2.0-flash", // Modello consigliato nel 2026
            contents: [{ role: "user", parts: [{ text: prompt }] }]
        });
        res.send({text:response.text});
    }
    catch (error) {
        console.error("Errore server:", error);
        res.status(500).send("erro.message")
    }
});
```

Lato client

```
btnInvia.addEventListener("click", async function(){
   const prompt = txtPrompt.value;
   if(prompt.trim() == ""){
      alert("Inserisci una domanda !")
      retrun
   }

   display.innerHTML = "searching ....."
   let httpResponse =await inviaRichiesta("POST", "/chat", {prompt} )
   if(httpResponse.status==200) {
      display.innerHTML = httpResponse.data.text
   }
   else
      alert(httpResponse.status + " : " + httpResponse.err)
})

document.addEventListener('keydown', function(event) {
    if (event.keyCode == 13)
         richiedi();
});
```

HTML

```
    <h1>Prova Google Gemini</h1>
    <input type="text" id="txtPrompt" placeholder="Chiedi qualcosa ...">
    <button id="btnInvia">Chiedi</button>
    <!-- pre-wrap non compatta gli spazi multipli -->
    <p id="display" style="white-space: pre-wrap;">
    </p>
```

Utilizzo diretto lato client

La libreria può essere utilizzata anche lato client, ma solo da sorgenti http e non da sorgenti file. L'utilizzo lato client è comunque sconsigliato perché non sicuro. Chiunque potrebbe rubare la chiave e consumare il credito. Linkata la libreria, la procedura è praticamente la stessa del lato server:

importmap

La libreria deve essere importata tramite uno `<script type="importmap">`
`importmap` (mappa delle importazioni) è una nuova funzionalità che permette di assegnare dei nomi (alias) per le varie URL utilizzando il seguente formato **`{"alias":URL}`** In pratica il browser: ogni volta che nel codice vedrà un certo nome nome 'X', andrà a scaricare il file dall'indirizzo 'Y'".

```
<script type="importmap">
{
    "imports": {
        "@google/genai": "https://esm.run/@google/genai"
    }
}
</script>
```

`<script type="importmap">` Indica al browser che il contenuto all'interno non è codice da eseguire subito, ma una tabella di riferimento per i futuri import. Deve essercene solo uno per pagina

`"imports": { ... }` È l'oggetto che contiene tutte le tue mappature

`"@google/genai"` è l'alias, cioè il nome che si userà nel codice JavaScript.
@google/genai è il nome ufficiale del pacchetto; in questo modo il codice sarà identico a quello che si potrebbe scrivere in un progetto lato server.

`"https://esm.run/@google/genai"` È l'indirizzo reale del CDN. esm.run è un servizio che ìtrasformaì le librerie ESM nate per Node.js in file ESM utilizzabili in javascript.

```
<script type="module" src="./index.js">  </script>
```

```
import { GoogleGenAI } from "@google/genai";
const API_KEY = "LA_TUA_API_KEI";
const genAI = new GoogleGenAI(API_KEY);

btnInvia.addEventListener("click", async function(){
   const prompt = txtPrompt.value;
   if(prompt.trim() == ""){
      alert("Inserisci una domanda !")
      return
   }

   display.innerHTML = "searching ....."
   try {
      const response = await genAI.models.generateContent({
         model: "gemini-2.0-flash", // Modello consigliato nel 2026
         contents: [{ role: "user", parts: [{ text: prompt }] }]
      })
      display.innerHTML = response.text
   }
   catch (error) {
      display.innerHTML = "Errore: " + error.message;
      console.error(error);
   }
})
```